K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

VANTA

AUTOMATED VULNERABILITY MANAGEMENT PLATFORM

Brief Project Documentation

Course	ENSP 465: New Age Programming Languages Lab
Semester / Section	5th Semester / Section A
Academic Year	2026–2027
Faculty Guide	Dr. Hemraj Kumawat
Student	Akansh Pandey | 2401830006 | B.Sc. (Hons.) Cybersecurity
Project Team	Aditya Chouhan (2401830001) and Kartikeya Drall (2401720014)

1. Project Overview

VANTA is a web-based vulnerability management platform that organizes security findings imported from network-scanning reports. It provides a centralized workspace to review findings, inspect discovered hosts and services, and track remediation status. The implemented workflow focuses on importing Nmap XML reports, processing their supported information, and presenting the resulting records through a dashboard and API.

2. Project Objectives

- Centralize security findings and scan-import information in one application.
- Accept and process Nmap XML reports from authorized assessments or sanitized test data.
- Preserve useful source context while organizing imported observations into structured records.
- Provide dashboard summaries, finding search/filtering, and remediation-status updates.
- Demonstrate an integrated application using React, Go, PostgreSQL, Docker, and cloud deployment.

3. Key Features

Feature	Description
Dashboard	Summarizes findings, recent activity, and import information.
Nmap XML import	Accepts an Nmap XML file and processes supported host, port, and service information.

Feature	Description
Findings management	Lists findings with search and severity/status filtering.
Status tracking	Allows a finding's remediation status to be updated and records status history.
Observations	Displays information associated with imported assets and services.

4. Technologies Used

Technology	Role in the project
React 18 + TypeScript	Builds the interactive frontend and dashboard.
Vite 5	Frontend development server and production build tool.
Go 1.22	Implements the backend HTTP API and application logic.
Chi Router	Routes HTTP requests to backend handlers.
PostgreSQL 16	Stores findings, imports, observations, and workflow data.
pgx / pgxpool	Provides Go-to-PostgreSQL connectivity and connection pooling.
Docker / Docker Compose	Containerizes services and supports repeatable local development.
GitHub	Hosts source code and version history.
Vercel + Render	Hosts the frontend on Vercel and the Go API and PostgreSQL on Render.

5. Implementation

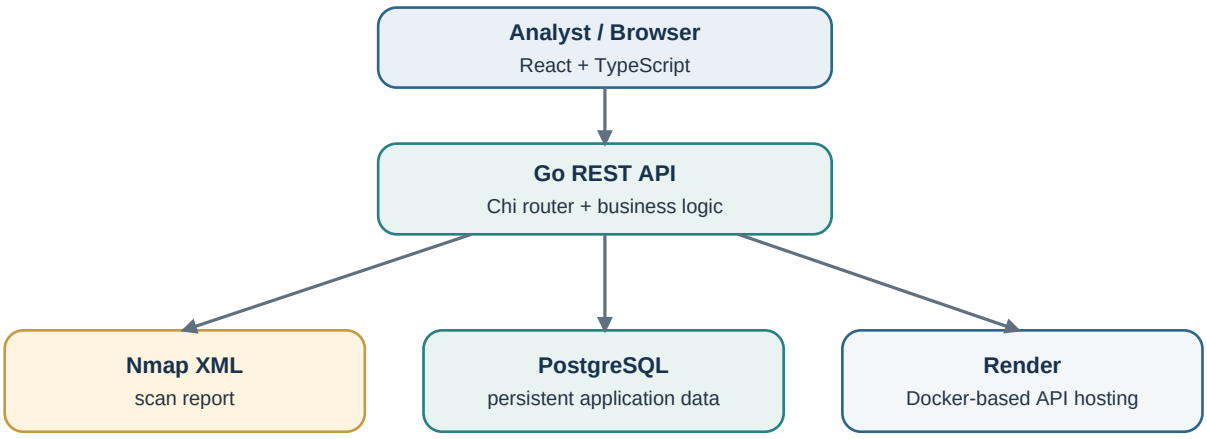
Frontend: The React and TypeScript single-page application provides the dashboard, findings views, filtering controls, and Nmap XML import interface. It communicates with the backend using HTTP requests through a shared API service.

Backend: The Go application uses the standard HTTP server and Chi router to expose REST endpoints. The API handles health/readiness checks, dashboard data, findings, status updates, Nmap imports, import history, and observations.

Data layer: PostgreSQL stores the application's persistent records. The backend connects through pgxpool, checks database readiness at startup, and applies SQL migrations before accepting requests.

Deployment: The frontend is deployed on Vercel. The Go backend is packaged using the existing Dockerfile and deployed as a Render web service. Render PostgreSQL is used as the production database. Environment variables keep deployment-specific connection details outside the source code.

6. System Architecture



Frontend is deployed on Vercel; it communicates with the API over HTTPS.

The browser loads the Vercel-hosted frontend. API requests travel over HTTPS to the Go service on Render, which reads and updates records in the managed PostgreSQL database. Nmap XML reports are supplied as input to VANTA; the platform imports reports rather than running the scanner itself.

7. Main Data Flow

Imported records remain linked to their source/import context.

A user uploads an Nmap XML report. The backend validates the request, parses supported report data, and stores the resulting records in PostgreSQL. The frontend then retrieves dashboard, finding, and observation data through the API. Users can review findings and update their remediation status.

8. API and Functional Modules

Endpoint	Purpose
GET /api/health	Checks whether the API process is responding.
GET /api/ready	Checks whether the PostgreSQL connection is available.
GET /api/dashboard	Returns dashboard metrics and recent records.
GET /api/findings	Lists findings with supported filters.
GET /api/findings/{id}	Returns an individual finding.
PATCH /api/findings/{id}/status	Updates a finding's status and remediation history.
POST /api/imports/nmap	Imports an Nmap XML file (multipart field: file).
GET /api/imports and /api/observations	Returns import history and imported observations.

9. Scope and Limitations

The current deployed implementation supports the Nmap XML import workflow and the listed dashboard, findings, observations, and status-management API operations. The original Phase I proposal also identified OWASP ZAP JSON ingestion and broader reporting/correlation capabilities as planned scope; these should not be described as completed unless separately implemented and verified. VANTA is a management and tracking platform, not a scanner: it does not itself scan targets, execute exploits, or automatically remediate vulnerabilities.

10. Conclusion

VANTA demonstrates an end-to-end vulnerability-finding management workflow using a modern web frontend, a Go REST API, and PostgreSQL persistence. Its deployed architecture separates the user interface, backend processing, and database, while providing a practical way to import Nmap scan information, review findings, and track remediation status.

Prepared for ENSP 465: New Age Programming Languages Lab | K.R. Mangalam University | Academic Year 2026–2027